

A watercolor illustration of a forest path. The path is a mix of yellow and brown, with scattered red leaves. It leads through a green forest with tall, thin trees. The style is soft and painterly.

HOWTO

CONTINUOUS PERFORMANCE ENGINEERING

Henrik Ingo
Fosdem 2026

NYRKIÖ

POLL

Continuous Performance Engineering
maturity levels

1. We don't really do benchmarking
2. Benchmarks yes, continuous no
3. Some benchmarks run in CI
...but we ignore the results
4. Benchmarks in CI,
Daily or more often,
Actionable results,
Regressions fixed within weeks

TYPICAL DISTRIBUTION OF ANSWERS (NORMAL DEVELOPERS)

50%

Simply don't care about
performance.
(Often rightly so!)

[Google trends](#)

TYPICAL DISTRIBUTION OF ANSWERS (NORMAL DEVELOPERS)

> 50%

< 50%

Simply don't care about
performance.
(Often rightly so!)

Benchmarking happens
Before release
One perf engineer
"Right Shift"

Benchmarks in CI
Noise 20% -100%
Alerts = distraction

Only a handful or
companies. 100?

CONTINUOUS INTEGRATION

CONTINUOUS TESTING

CONTINUOUS DEPLOYMENT

DEVOPS

LEFT SHIFTING

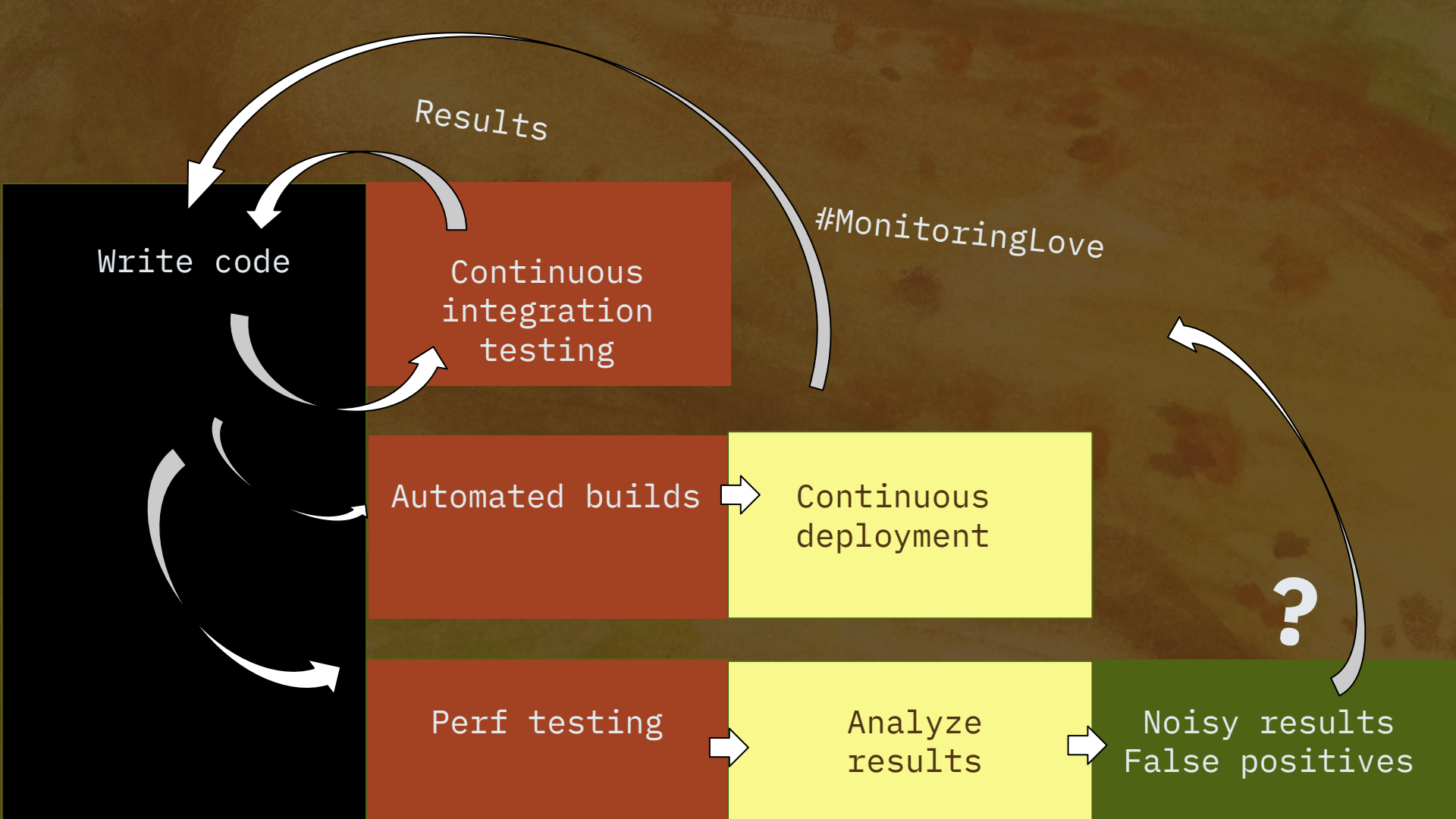
ITERATIONS

CONTINUOUS PERFORMANCE ENGINEERING?

Somewhere in that evolution, they forgot to bring the performance engineer



Picture: Velociraptor, the fastest dinosaur



WHY WAS PERFORMANCE ENGINEERING LEFT IN 1999?

Deploying is necessary - benchmarking is optional

Performance Engineers busy fixing prod / customer problems

....and maybe enjoying it?

Math is hard, but we're gonna need it

Relevant tuning is unintuitive, not well known

CONTINUOUS PERFORMANCE ENGINEERING

CONTINUOUS PERFORMANCE ENGINEERING IS POSSIBLE

CONTINUOUS PERFORMANCE ENGINEERING
~~IS POSSIBLE~~
IS ACHIEVABLE!

I WILL TELL YOU HOW

I WILL TELL YOU HOW

IF YOU PROMISE TO TELL EVERYONE ELSE?

POLL

The community is so new,
we don't even have consensus
for what to call it?

1. Continuous Performance Engineering
2. Continuous Benchmarking
3. DevPerfOps
4. Something else?
5. How should I know, I'm hearing about this for the first time....

WHO ELSE TALKS ABOUT THIS?

Continuous Benchmark Actions

GitHub Action for Continuous Benchmarking

This repository provides a [GitHub Action](#) for continuous benchmarking. If your project has some benchmarking, this action collects data from the benchmark outputs and monitor the results on GitHub Actions workflow runs.

- This action can store collected benchmark results in [GitHub pages](#) branch and provide a chart of the results are visualized on the GitHub pages of your project.

Apache Otava

Change Detection for Continuous Performance Engineering

: Netherlands

Business Description:

Perfana delivers software performance engineering solutions to deliver high-performing software.

Based in: Amsterdam



NYRKİÖ

Dashboards

Product

Docs

Get Started

What is CodSpeed?

Integrated CI tools for software engineering to deliver the next delivery on performances.

{codspeed 🚀}

Documentation Guides </> API Reference

Discord

Get Started

Quickstart



Subscribe

@ScottMooreConsultingLLC



@ScottMooreConsultingLLC

Subscribe



ICPE

ACM/SPEC International Conference on Performance Engineering

NYRKİÖ

AGENDA

1. Benchmarking
 - Tools
 - Process
 - Agile Benchmarking
2. Change Point Detection
 - Where it all started!
 - Apply Math & Science
 - Demo Tool
3. Minimizing noise in the Benchmarks
 - Core principles, again
 - Some learnings
 - 10 years later

A watercolor illustration of a forest scene. A dirt path, covered with fallen brown and orange leaves, leads from the bottom left towards the center of the image. The path is flanked by green grass and patches of yellow. In the background, several tall, slender trees with green foliage stand against a bright, hazy sky. The overall style is soft and painterly.

BENCHMARKING PROCESS AND TOOLS

ON BENCHMARK TOOLS & DESIGN

This topic was already covered in a previous talk by Kemal Akkoyun & Augusto de Oliveira. In the interest of time, I will talk about something else.

But in general, at this point each language has its own standard benchmark framework: Java & JMH, Python & pytest-benchmark, etc.

Frameworks to run fully, end to end automated distributed benchmarks exist. Personally I believe we will see new innovation coming from here.

Ex: K8s cluster on a 128 CPU server, simulated network latency.

CORE PRINCIPLES (PART 1)

Sharing* is caring

Writing** is powerful

When writing, including at least one graph is mandatory

*) Benchmark results, findings

**) Or present at conferences

CORE PRINCIPLES (FOR CONTINUOUS BENCHMARK DESIGN)

You get what you measure

Basic scientific process: Hypothesize -> Configure -> Test -> Analyze -> GOTO 1

Only change one thing per iteration

CORE PRINCIPLES (FOR CONTINUOUS BENCHMARK DESIGN)

It is true that benchmarks should be represent **realistic customer workloads**

However, that can take weeks, or months, just for a single benchmark!

That is not agile.

One simple benchmark running in CI is more valuable than the perfect benchmark you didn't do yet

Simple benchmarks are easier to analyze and reason about

For Continuous Benchmarking, having even one benchmark running in CI, allows you to start tracking it's history with graphs and change point detection!

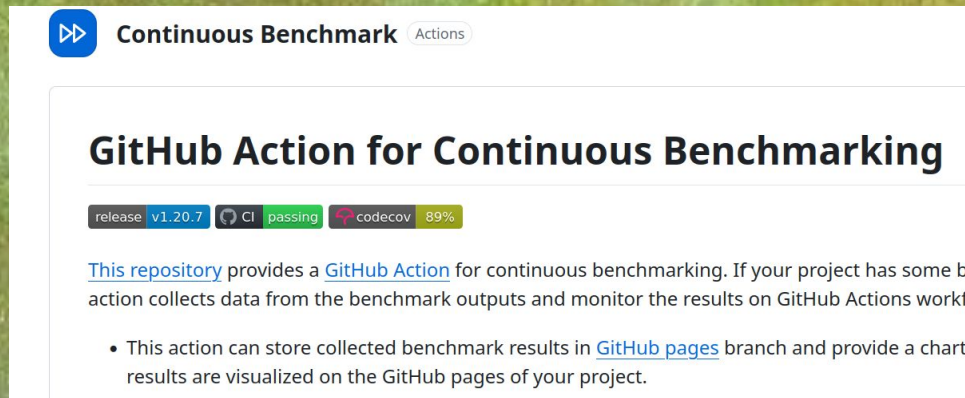
GITHUB-ACTION-BENCHMARK

Use in your workflows immediately after the benchmark step

Supports output from all major frameworks

Stores result history in your github repo.

Uses simple threshold based
Alerting. By default, starts alerting
from regression being
100% or more. (2x slowdown)



The screenshot shows the GitHub repository page for 'Continuous Benchmark' by 'Actions'. The repository name is 'Continuous Benchmark' and it is categorized under 'Actions'. The repository has a 'release' tag for 'v1.20.7', a 'CI' workflow status of 'passing', and a 'codecov' badge showing '89%'. The repository description states: 'This repository provides a GitHub Action for continuous benchmarking. If your project has some benchmarking action, this action collects data from the benchmark outputs and monitor the results on GitHub Actions workflow runs.' A bullet point indicates: 'This action can store collected benchmark results in [GitHub pages](#) branch and provide a chart of the results are visualized on the GitHub pages of your project.'

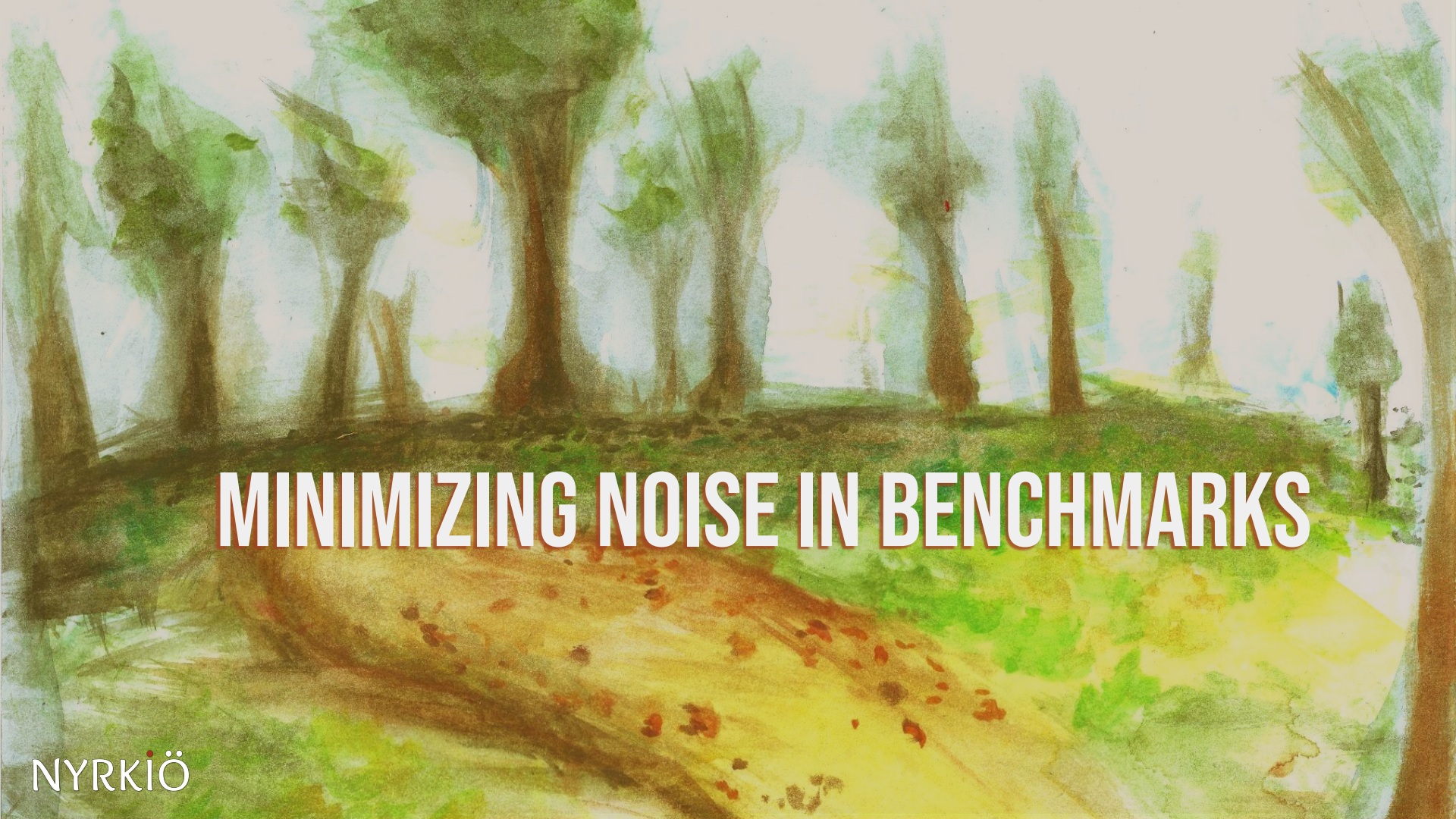


CHANGE POINT DETECTION

CORE PRINCIPLES (FOR CHANGE POINT DETECTION)

Math is hard*.

*) Math is unfortunately necessary.

A watercolor illustration of a forest scene. A dirt path with scattered red and brown leaves leads from the bottom center towards the background. The path is flanked by green grass and patches of yellow. Several tall, slender trees with green foliage stand in the background, their trunks rendered in shades of brown and grey. The overall style is soft and painterly, with visible brushstrokes and a gentle color palette.

MINIMIZING NOISE IN BENCHMARKS

CORE PRINCIPLES (FOR SERVER TUNING)

Don't believe everything you read on the internet

Assume nothing. Measure everything.

Apply the scientific process